

# A Zero-Knowledge Proof of Possession

for the Agda-in-Lean proof system  
protocol version `agda-zkpop-v1`

2026-08-26

## Abstract

This document is evidence that its author holds a specific, unpublished bundle of 115 source artefacts — an Agda type checker ported to Lean together with the self-hosted Agda core it checks — and that the author knows the secret derived from that bundle. It contains no source code from the bundle. The evidence is a commitment to the bundle, a non-interactive zero-knowledge proof of knowledge of an opening of that commitment, and the openings of 8 positions chosen by the proof's own challenge, not by the author. The protocol is stated and proved — completeness, knowledge soundness by extraction, and *perfect* honest-verifier zero knowledge — in Lean 4 with Mathlib, and the concrete numbers printed here are re-checked by the Lean kernel, including the SHA-256 hashing.

## 1 The claim

Let  $B$  be a bundle of 115 files: the 94 Lean modules of the checker and the 21 Agda modules of the core it checks, 696 104 bytes in total. The claim is:

*The author holds  $B$ , whose salted Merkle commitment is the root  $R$  printed in Appendix A; the author knows the scalar  $m = m(B)$  derived from the complete opening of that commitment; and the 8 artefacts opened in Appendix B, whose positions the challenge chose, are genuine members of the committed bundle.*

Nothing in this document reveals a line of the bundle's contents. What is disclosed is listed in Section 2, exactly and exhaustively.

The claim data hashed into the proof is

```
{"agda_modules":21,"anchors":{"README.md":"dcb7d2c923e6a00f7496a7079a8ff39ad0f7a5aa2d9c70c00cf35f88e58b8479","SELF-HOSTING.md":"494ac1723f104989483f8aa3d7962b240462599b6a65b157480c9b340d257cd5","STATUS.md":"7aebbb0bff071902859037b3134b95a52b09e40864227b526199f5eff7314b9e","introspection/shhs/catalog.json":"0721db0853c4d87dd2bbcf69201332766418a9dac60f69480e7d57450ee8a605"},"artefact_count":115,"lean_modules":94,"opened_count":8,"subject":"Agda-to-Lean self-hosting proof system (this repository)","total_bytes":696104,"tree_depth":7,"tree_leaves":128,"version":"agda-zkpop-v1"}
```

## 2 What is disclosed, and what is not

| Disclosed                                            | Not disclosed                         |
|------------------------------------------------------|---------------------------------------|
| the number of artefacts, and how many are Lean/Agda  | any artefact's contents               |
| their total size in bytes                            | the contents of any unopened record   |
| the Merkle root $R$ of the salted commitments        | the salts of unopened positions       |
| the parameters, $Y$ , and $(a, c, z_1, z_2)$         | the secret scalars $m, r, u, s$       |
| for 8 chosen positions: path, size, SHA-256          | the same data at every other position |
| SHA-256 digests of four public description documents | the documents' contents               |

The last column is not a matter of good intentions. For the transcript it is a theorem: the distribution of  $(a, c, z_1, z_2)$  for a fixed challenge is *identical* to the output of a simulator that never sees the witness (Section 4, `hvzk_uniform`). For the unopened leaves it is a theorem under the stated hiding assumption on the salted leaf commitment: two bundles that agree on the opened positions induce the same distribution of the entire published view (`view_hiding`).

### 3 The protocol

#### 3.1 Hashing convention

Every hash call is domain-separated and length-prefixed:

$$H(d; x_1, \dots, x_k) = \text{SHA-256}(\text{be}_{32}(|d|) \| d \| \text{be}_{32}(|x_1|) \| x_1 \| \cdots \| \text{be}_{32}(|x_k|) \| x_k),$$

so two different tuples never share a hash input. The domains used are `agda-zkpop-v1/record`, `/leaf`, `/pad`, `/node`, `/bundle`, `/scalar`, `/challenge`, `/select` and `/generator`.

#### 3.2 Group parameters

A Schnorr group: a 2048-bit prime  $p$ , a 256-bit prime  $q$  dividing  $p - 1$ , and  $g$  of order  $q$  modulo  $p$ . The second generator is derived by hashing,

$$h = (H(\text{agda-zkpop-v1/generator}; p, q, g, \text{ctr}))^{(p-1)/q} \bmod p, \quad \text{ctr} = 0,$$

the smallest counter giving  $h \neq 1, g$ . Nobody, the author included, knows  $\log_g h$ ; that is what makes the Pedersen commitment below binding.

#### 3.3 Committing the bundle

Sort the artefacts by path. For artefact  $i$  with path  $\pi_i$ , size  $s_i$  and contents  $c_i$ , the *record* and the *leaf* are

$$\rho_i = H(\text{/record}; \pi_i, \text{beg}(s_i), \text{SHA-256}(c_i)), \quad \ell_i = H(\text{/leaf}; \text{beg}(i), \sigma_i, \rho_i),$$

where  $\sigma_i$  is 32 fresh random bytes. The leaves are padded to 128 with  $H(\text{/pad}; \text{beg}(i), \sigma_i)$  and hashed pairwise with  $H(\text{/node}; \cdot, \cdot)$  into a perfect binary tree of depth 7 with root  $R$ .

The salt  $\sigma_i$  is what makes the commitment hiding: without it, a leaf is a guessable function of a file, and a reader could test candidate files against the tree.

#### 3.4 The secret and its Pedersen commitment

The bundle secret is the digest of the *complete* opening of the tree,

$$m = H(\text{/scalar}; R, H(\text{/bundle}; \sigma_1 \| \rho_1, \dots, \sigma_n \| \rho_n)) \bmod q,$$

so producing  $m$  requires every salt and every record — that is, the whole bundle. With uniform  $r \in \mathbb{Z}_q$  the author publishes  $Y = g^m h^r \bmod p$ .

#### 3.5 The proof of knowledge

The published transcript is the Fiat–Shamir compilation of the standard  $\Sigma$ -protocol for knowledge of an opening of a Pedersen commitment:

|            |                                                                        |
|------------|------------------------------------------------------------------------|
| commit:    | $u, s \leftarrow \mathbb{Z}_q, \quad a = g^u h^s \bmod p,$             |
| challenge: | $c = H(\text{/challenge}; p, q, g, h, Y, a, R, \text{claim}) \bmod q,$ |
| respond:   | $z_1 = u + c m, \quad z_2 = s + c r \pmod{q},$                         |
| verify:    | $g^{z_1} h^{z_2} \equiv a Y^c \pmod{p}.$                               |

Because the challenge is a hash of the first message *and* of the commitment  $R$ , the transcript is bound to this bundle and could not have been prepared for a different one, nor adapted after seeing  $c$ .

### 3.6 Challenge-selected openings

The same challenge selects which positions are opened:  $\text{idx}_t = \text{H}(\text{select}; c, \text{beg}(t)) \bmod n$  for  $t = 0, 1, 2, \dots$ , skipping repeats, until 8 distinct positions are chosen. The author reveals for each the salt, the path, the size, the digest of the contents, and the Merkle path to  $R$ . Since  $c$  depends on  $R$  and on  $a$ , both fixed before  $c$  exists, the author cannot steer the choice towards positions that happen to be well-formed.

## 4 What is proved, and where

Everything in this section is machine-checked Lean 4 (with Mathlib) in the repository that holds the bundle; no statement below uses `sorry` or any axiom beyond Lean’s own `propext`, `Classical.choice` and `Quot.sound`.

| Theorem                               | File                  | Content                                                                                 |
|---------------------------------------|-----------------------|-----------------------------------------------------------------------------------------|
| ZKPoP.completeness                    | ZKPoP/Sigma.lean      | an honest transcript always verifies                                                    |
| ZKPoP.special_soundness               | ZKPoP/Sigma.lean      | two transcripts, one first message, two challenges $\Rightarrow$ the opening, extracted |
| ZKPoP.hvzk_uniform                    | ZKPoP/Sigma.lean      | honest and simulated transcript distributions are <i>equal</i>                          |
| ZKPoP.ni_completeness                 | ZKPoP/FiatShamir.lean | the non-interactive proof verifies                                                      |
| ZKPoP.ni_challenge_binding            | ZKPoP/FiatShamir.lean | the challenge is fixed by $(Y, a)$                                                      |
| ZKPoP.ni_zero_knowledge               | ZKPoP/FiatShamir.lean | simulation in the programmable random-oracle model                                      |
| ZKPoP.leaf_hiding                     | ZKPoP/Bundle.lean     | a salted leaf leaks nothing                                                             |
| ZKPoP.view_hiding                     | ZKPoP/Bundle.lean     | the published view leaks nothing beyond the opened positions                            |
| ZKPoP.root_hiding                     | ZKPoP/Bundle.lean     | in particular the root leaks nothing                                                    |
| SHHS.verifyPath_pathFor               | SHHS/Merkle.lean      | Merkle completeness                                                                     |
| SHHS.leaf_of_verifyPath               | SHHS/Merkle.lean      | Merkle soundness, under an explicit injectivity hypothesis on the hash                  |
| ZKPoP.powMod_eq                       | ZKPoP/Arith.lean      | the fast modular exponentiation is $b^e \bmod m$                                        |
| ZKPoP.Sha256.sha256_abc <i>et al.</i> | ZKPoP/Sha256.lean     | the implementation matches the FIPS 180-4 test vectors                                  |

The concrete numbers printed in this document are not merely asserted: they are theorems, closed by kernel computation.

| Theorem                                     | Content                                                                                                                                          |
|---------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| Instance.g_pow_q, h_pow_q, Y_pow_q, a_pow_q | every published group element has order dividing $q$                                                                                             |
| Instance.q_dvd_p_sub_one                    | $q \mid p - 1$                                                                                                                                   |
| Instance.sigma_verifies                     | $g^{z_1} h^{z_2} \equiv a Y^c \pmod{p}$                                                                                                          |
| InstanceCommit.openings_verify              | each opened leaf is the salted commitment to its record, and its path of depth 7 reproduces $R$                                                  |
| InstanceCommit.positions_correct            | the opened positions are the ones $c$ selects                                                                                                    |
| InstanceCommit.challenge_correct            | $c$ is the hash of $(p, q, g, h, Y, a, R, \text{claim})$                                                                                         |
| Instance.published_verifies                 | the published numbers name elements of the deployed group, and the transcript satisfies the abstract predicate <code>ZKPoP.Verifies</code> there |
| Instance.published_knowledge                | assuming $q$ prime: a second accepting transcript with the same first message yields the opening                                                 |

Two remarks on what that means. First, these are *kernel* computations: SHA-256 is implemented in Lean (`ZKPoP/Sha256.lean`) and evaluated by the kernel on the published bytes, so verifying the commitment side of this document needs no tool outside Lean. Second, the abstract and the concrete sides genuinely meet: `ZKPoP/Deploy.lean` proves that the  $q$ -th roots of unity in  $\mathbb{Z}_p$  form a  $\mathbb{Z}_q$ -module — the structure the abstract theorems are stated over — and that the published numbers name elements of it whose transcript satisfies `ZKPoP.Verifies`. So completeness, extraction and perfect zero knowledge are theorems *about this transcript*, not about a protocol resembling it. (That construction needs no primality:  $q$ -torsion is a subgroup of the units of  $\mathbb{Z}_p$  whatever  $p$  is. Primality of  $q$  enters only in extraction, and it enters as a hypothesis.)

## 5 Assumptions

The proof rests on assumptions that no formalisation can discharge; they are listed here rather than buried.

1. **Discrete logarithms are hard** in the order- $q$  subgroup of  $\mathbb{Z}_p^*$  ( $p$  of 2048 bits,  $q$  of 256). Without this,  $m$  could be computed from  $Y$  and “knowledge of  $m$ ” would mean nothing. This is why  $\log_g h$  must be unknown, and why  $h$  is derived by hashing.
2. **SHA-256 behaves like a random oracle** — for the Fiat–Shamir challenge (so it cannot be steered), for the selection of opened positions, and for the hiding of salted leaves. The Lean hiding theorems are stated *relative* to that idealisation (`PerfectHiding`), not on top of an unproved claim about SHA-256 itself.
3. **SHA-256 is collision-resistant**, for the binding of the Merkle tree. The Lean Merkle soundness theorem takes injectivity of the compression function as an explicit hypothesis, for the same reason.
4.  **$p$  and  $q$  are prime**. This is checked by Miller–Rabin with fixed small bases and 64 random bases in `scripts/zkpop_verify.py`; it is *not* proved in Lean, and is recorded here as an assumption. The Lean extraction theorem takes primality of  $q$  as a hypothesis.
5. **The salts were sampled uniformly at random** by the author, from the operating system’s CSPRNG, and  $r$ ,  $u$ ,  $s$  likewise. A verifier cannot check this; a prover who reused  $u$  across two different challenges would leak  $m$ , by the very extraction theorem that makes the proof meaningful.

## 6 What this does and does not establish

It establishes that the author holds a bundle matching the published root: to produce  $m$  one needs every salt and every record, and to answer the challenge one needs  $m$  (or the discrete logarithm). It establishes that the opened artefacts — chosen by the proof, not by the author — are members of that committed bundle. And it establishes that a reader learns nothing else: the transcript is perfectly simulatable, and the unopened leaves are distributionally independent of their contents.

It does not, by itself, establish that the artefacts *constitute a working proof system*. That is a claim about the contents, and proving it in zero knowledge would require a circuit for the type checker’s own execution. The document is honest about the split: possession and integrity are proved here; what the possessed system does is described by the four anchor documents whose digests appear in Appendix B, and can be confirmed by anyone who is later given the bundle — the root ties the disclosure to exactly the bundle committed to today.

## 7 How to verify

From this document alone, with the values in Appendix A and B, and nothing but a SHA-256 implementation and big-integer arithmetic:

1. check  $q \mid p - 1$ , that  $p, q$  are prime, and that  $g^q \equiv h^q \equiv Y^q \equiv a^q \equiv 1 \pmod{p}$ ;
2. recompute  $h$  from  $p, q, g, \text{ctr}$  by the published derivation;
3. recompute  $c$  as the hash of  $(p, q, g, h, Y, a, R, \text{claim})$ ;
4. check  $g^{z_1} h^{z_2} \equiv a Y^c \pmod{p}$ ;
5. recompute the selected positions from  $c$  and check they are the opened ones;
6. for each opened position, recompute the record, the leaf, and fold its Merkle path to  $R$ .

With the repository that accompanies this protocol, all of the above is

```
python3 scripts/zkpop_verify.py
lake build RequestProject.ZKPoP
```

the first re-running the checks in Python, the second re-running the commitment and transcript checks inside the Lean kernel and re-checking every theorem of Section 4. A holder of the bundle can add `--repo .` to the first command to confirm the opened artefacts against the files on disk.

## A Published values

Digits are grouped in tens purely for legibility. The SHA-256 of the machine-readable form of everything below (zkpop/public/statement.json) is

2e254090454bfa39 6d5e8eeb4a177540 9206a6e202022e3a 934b46f9a8c1fced

### Merkle root $R$ (hex)

e38f28538a99edf3 188bbf21b8e5f093 23dede6a60027445 987ae09dac323949

$p$

1759002906 7608184719 0395526039 9170113743 1428891029 9606687040 8431151507 8704960583 5317347230  
8991744746 3286675505 2064466414 6205777746 8691926509 3004842942 2361360719 6935600643 0132245347  
1107757422 3705291144 2659324647 6655693770 1220692462 7370544752 2068888792 4036393587 5023085448  
8395105079 8466277952 1537608433 6485015556 0159241871 1080853172 5958265994 9630351109 2052305857  
8735624415 6434788788 9801651249 2957471898 8701405873 6625157856 7276482162 9373352793 2183874570  
4896880824 6125258520 8282039063 7506481069 4650745435 3975374817 2429260945 8656570592 2574825758  
0156500781 7413540654 2037815532 4287538574 9104006178 5348683830 5290773203 2246559

$q$

8045575838 4176723884 6396405944 4130246614 7133392892 0867653099 9819658946 5517831

$g$

7052519720 8513965909 5518182388 7605526505 0543499136 1587436103 5556230403 8862707366 2890657476  
9897292729 3409997763 6769339690 1984184439 0053305072 4697222181 3893691650 4444978125 6820050440  
4569499920 7234339072 9252904068 6566699955 6767620088 0135686302 6296493721 5333989631 7793335810  
7314544413 4259777728 8878698065 8818598119 0347737057 1049664983 0964683295 0867795458 5068788969  
2362296723 7987322192 9382377234 0512333997 8862501711 6687594352 0234684661 4758592536 8442035920  
2241972972 7732951348 0306473871 8041919930 1438277647 3573386947 2637855356 6855792001 3727470806  
4061426800 9686118978 4042004685 5472944455 3759373219 7504486421 3166716185 79921

$h$

```
4136398459 5738643538 3345223520 5658960255 2711284760 7263897843 9150380978 2762770674 1804753505
6360881730 2898788670 8331018590 7355560034 3258214235 5465543528 4098714615 2710588746 5296533262
7179947036 3159855358 9799406014 4402873078 1756693545 1722697734 5925642153 2011797631 3997117728
6392597570 5541874663 3840844903 6930773557 5949164625 8593331780 2354122916 8216338005 9453161878
3062286318 0610458563 4320824436 3627145037 2601292795 4468475452 0464100966 7506938273 2151088475
5396038316 0229813832 3078891704 6187608522 0686963243 1968997392 5461449327 5896052545 2254309423
1636801295 9233187288 4691677119 1752901323 8208062720 8690157929 2911527649 617508
```

$Y$

```
9605162838 9785022783 8459930194 0023510139 9518152364 3966765408 6092409234 0539782098 6603216498
7697690262 4796737153 6039958461 3802769211 8375538299 9799332841 7505298208 7066820520 9354944198
0541279715 7168430285 9941016343 5843730149 4338188760 7311609370 2344629503 8768887716 3118126110
8341732691 6237810792 2608761488 2720874521 3209277775 4983288705 9203707889 1509489036 5422554040
2134730127 4978579813 9477793002 2268468736 8880675663 2181609314 8793636398 8547216318 7816754240
5966897820 1782962867 9347852377 6821970547 9765328607 2786813808 3544426626 0781518131 4616074778
1872456968 3613454726 8134299220 6353839418 0283897250 1727319847 1503022340 164679
```

$a$

```
1736488040 8365995744 4681616226 0117001801 1957716150 3215531616 3522323321 7618546293 9208247277
3213602264 1774700007 9573052512 1458086403 3847421493 7025378393 4082234711 4920871994 5675551744
8032814079 9365553510 4157207322 1585901279 1859246804 5308310502 3122504334 3034533339 7062950809
5531869885 4366454877 4232035821 9112692286 8726582829 7701282853 2593212702 3331554558 1716774825
3763370993 6617726986 9125146606 9231490867 2194319297 9482005267 1006533945 2981284424 6671816127
8812174268 4735238039 0083926717 2267216755 5387874669 8402973898 0013314720 4208619929 0669107590
7414616437 4639641301 6800920410 0822902849 7362884258 3039888399 0868542057 1023235
```

$c$

```
2048515315 7433717966 4507333254 6907617200 0161934447 0699815996 0195086339 4900216
```

$z_1$

```
6177244031 5845862631 6436592998 0901600556 7733634093 4492252659 5075565945 6598566
```

$z_2$

```
6227357155 4191546393 1901030671 8507801523 0456342107 4234936278 5010377828 1895324
```

## B The openings

The 8 positions the challenge selected, with the artefact record revealed at each. The Merkle paths (7 sibling digests each) are in the machine-readable statement file; they are what the Lean theorem `openings_verify` folds to  $R$ .

| #   | path                                                  | bytes | SHA-256 of contents |
|-----|-------------------------------------------------------|-------|---------------------|
| 14  | RequestProject/Agda/Syntax/Concrete.lean              | 10041 | 00c8387e5891...     |
| 20  | RequestProject/Agda/Syntax/Internal/Base.lean         | 2461  | a2ddb0361128...     |
| 32  | RequestProject/Agda/Syntax/Pretty.lean                | 6229  | fcbaaa9c91a...      |
| 58  | RequestProject/Agda/TypeChecking/Monad/Base.lean      | 4730  | b0a620fd093e...     |
| 62  | RequestProject/Agda/TypeChecking/Monad/Signature.lean | 10616 | aef3331c3216...     |
| 85  | RequestProject/SHHS/Catalog.lean                      | 75139 | a9f22085fd1a...     |
| 102 | library/Agda/Core/Show.agda                           | 4209  | ee01e710a281...     |
| 108 | library/Agda/List.agda                                | 2379  | ecb8a73e52b3...     |

## Salts of the opened positions (hex)

| #   | salt                                                                 |
|-----|----------------------------------------------------------------------|
| 14  | 8ecaa0ba9bbc80af 01df1202344e2c6a 6eae12046d787651 63816bc18a597df3  |
| 20  | f90588bf3d278f98 47b6d710543fab71 a513dc6401ba0b09 bac56f341a405f72  |
| 32  | ceafebbcaac8c5d0 9e2ba8b5b836c293 63dcfdbba38766326 35ba8a0d747cc878 |
| 58  | 584570b5034e364c 3b6d25e9d52a06b5 f8d0e38cb2b3b0b5 e8f8c23e88a5778c  |
| 62  | 3875a033a4814ca7 0c05c417bc28cec3 0467084a6f5a6b14 a5d04f4eeb1b72e8  |
| 85  | 87481b14a2ab866b ee031a438e060b87 288b9751e198991b 8a6f931e9171529c  |
| 102 | 7f3ab7d09d7d19f5 46cf107e7d075eb4 5991b25eeaa38ae1 d748761af43859a2  |
| 108 | eb2b0f3977033524 3a64408f3018a90c a479bc1ebd8b4992 fd9f9fd427da6d4e  |

## Public anchors

Digests of the documents that describe the possessed system. They are not part of the bundle and are not opened by this proof; they let a verifier who is later given the repository confirm that it is the system this proof was made about.

| document                        | SHA-256                              |
|---------------------------------|--------------------------------------|
| README.md                       | dc7d2c923e6a00f7496a7079a8ff39a...   |
| SELF-HOSTING.md                 | 494ac1723f104989483f8aa3d7962b24...  |
| STATUS.md                       | 7aebbb0bfff071902859037b3134b95a5... |
| introspection/shhs/catalog.json | 0721db0853c4d87dd2bbcf6920133276...  |